

JOBSHEET X

QUEUE



Disusun oleh:

Nama : Galih Candra Kirana
No. Absen : 10
Kelas : TI 1G
NIM : 254107020080

POLITEKNIK NEGERI MALANG
Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur 65141
TAHUN 2025-2026

10.1 Tujuan Praktikum

1. Setelah melakukan materi praktikum ini, mahasiswa mampu: 1. Mengenal struktur data Queue
2. Membuat dan mendeklarasikan struktur data Queue 3. Menerapkan algoritma Queue dengan menggunakan array

10.2 Percobaan 1: Mahasiswa Mengumpulkan Tugas

10.2.1 Langkah-langkah Percobaan

Class Queue:

```
package Jobsheet10.Kode;

public class Queue {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue(int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }
}
```

```
public boolean IsFull() {  
    if (size == max) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

```
public void peek() {  
    if (!IsEmpty()) {  
        System.out.println("Elemen terdepan: " + data[front]);  
    } else {  
        System.out.println("Queue masih kosong");  
    }  
}
```

```
public void print() {  
    if (IsEmpty()) {  
        System.out.println("Queue masih kosong");  
    } else {  
        int i = front;  
        while (i != rear) {  
            System.out.println(data[i] + " ");  
            i = (i + 1) % max;  
        }  
        System.out.println(data[i] + " ");  
        System.out.println("Jumlah elemen = " + size);  
    }  
}
```

```
public void clear() {  
    if (!IsEmpty()) {
```

```

        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

```

```

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}

```

```

public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        dt = data[front];
    }
}

```

```

        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
            } else {
                front++;
            }
        }
    }
    return dt;
}
}

```

Class QueueMain:

```

package Jobsheet10.Kode;

import java.util.Scanner;

public class QueueMain {
    public static void menu() {
        System.out.println("Masukkan operasi yang diinginkan: ");
        System.out.println("1. Enqueue");
        System.out.println("2. Dequeue");
        System.out.println("3. Print");
        System.out.println("4. Peek");
        System.out.println("5. Clear");
        System.out.println("-----");
    }

    public static void main(String[] args) {

```

```
Scanner sc = new Scanner(System.in);

System.out.print("Masukkan kapasitas queue: ");
int n = sc.nextInt();

Queue Q = new Queue(n);

int pilih;
do {
    menu();
    pilih = sc.nextInt();

    switch (pilih) {
        case 1:
            System.out.print("Masukkan data baru: ");
            int dataMasuk = sc.nextInt();
            Q.Enqueue(dataMasuk);
            break;

        case 2:
            int dataKeluar = Q.Dequeue();
            if (dataKeluar != 0) {
                System.out.println("Data yang dikeluarkan: " + dataKeluar);
            }
            break;

        case 3:
            Q.print();
            break;

        case 4:
            Q.peek();
```

```

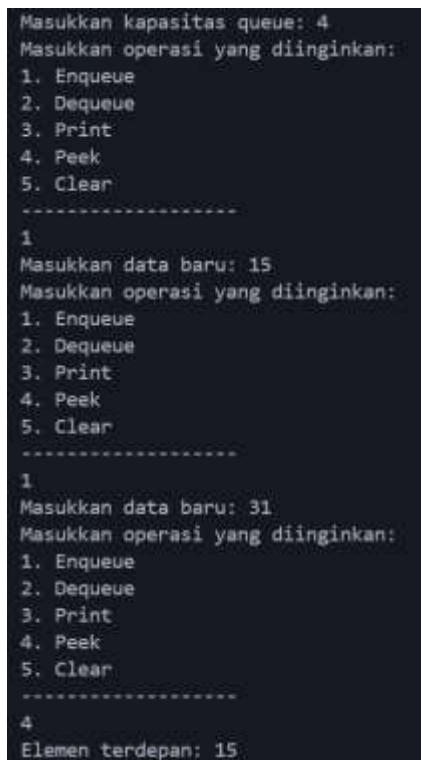
        break;

        case 5:
            Q.clear();
            break;
    }
} while (pilih >= 1 && pilih <= 5);

sc.close();
}
}

```

10.2.2 Verifikasi Hasil Percobaan



```

Masukkan kapasitas queue: 4
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 31
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
4
Elemen terdepan: 15

```

10.2.3 Pertanyaan!

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?

Jawaban:

Nilai pada atribut size bernilai 0 karena menandakan bahwa elemen di dalam array masih kosong, sedangkan atribut front dan rear bernilai -1 sebagai penanda bahwa belum ada elemen yang menempati indeks depan dan belakang array.

2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {  
    rear = 0;  
}
```

Jawaban:

Potongan kode tersebut digunakan untuk mengecek apakah posisi rear sudah berada di indeks terakhir array. Jika iya, maka nilai rear akan di reset ke 0.

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {  
    front = 0;  
}
```

Jawaban:

Potongan kode tersebut digunakan untuk mengecek posisi front sudah berada di indeks terakhir array. Jika iya, maka nilai front akan di reset ke 0.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?

Jawaban:

Pada method print, variabel i dimulai dari front karena elemen pertama yang harus ditampilkan adalah elemen terdepan dalam queue, bukan selalu elemen pada indeks 0.

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

Jawaban:

Kode ini berfungsi agar variabel i bergerak ke indeks berikutnya dan kembali ke awal array jika sudah mencapai batas akhir.

6. Tunjukkan potongan kode program yang merupakan queue overflow!

Jawaban:

```
if (IsFull()) {  
    System.out.println("Queue sudah penuh");  
}
```

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawaban:

Class Queue:

```
public int Enqueue(int dt) {  
    if (IsFull()) {  
        System.out.println("Queue sudah penuh");  
        return -1;  
    } else {  
        if (IsEmpty()) {
```

```

        front = rear = 0;
    } else {
        if (rear == max - 1) {
            rear = 0;
        } else {
            rear++;
        }
    }
    data[rear] = dt;
    size++;
    return 0;
}
}

public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
        return -1;
    } else {
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
            } else {
                front++;
            }
        }
    }
    return dt;
}
}

```

Class Main:

case 1:

```

    System.out.print("Masukkan data baru: ");
    int dataMasuk = sc.nextInt();
    int procsMasuk = Q.Enqueue(dataMasuk);
    if (procsMasuk == -1) {
        System.out.println("Keluar.");
        pilih = 0;
    }
    break;

```

```

case 2:
    int dataKeluar = Q.Dequeue();
    if (dataKeluar == -1) {
        System.out.println("Keluar.");
        pilih = 0;
    }
    if (dataKeluar != 0) {
        System.out.println("Data yang dikeluarkan: " + dataKeluar);
    }
    break;

```

10.3 Percobaan 2: Konversi Nilai Tugas ke Biner

10.3.1 Langkah – langkah percobaan

Class Mahasiswa:

```

package Jobsheet10.P2;

public class Mahasiswa {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
}

```

Class LayananAkademik:

```
package Jobsheet10.P2;
```

```
public class AntrianLayanan {
```

```
    Mahasiswa[] data;
```

```
    int front;
```

```
    int rear;
```

```
    int size;
```

```
    int max;
```

```
    public AntrianLayanan(int max) {
```

```
        this.max = max;
```

```
        data = new Mahasiswa[max];
```

```
        front = 0;
```

```
        rear = -1;
```

```
        size = 0;
```

```
    }
```

```
    public boolean IsEmpty() {
```

```
        if (size == 0) {
```

```
            return true;
```

```
        } else {
```

```
            return false;
```

```
        }
```

```
    }
```

```
    public boolean IsFull() {
```

```
        if (size == max) {
```

```
            return true;
```

```
        } else {
```

```

        return false;
    }
}

public void lihatTerdepan() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.println("Mahasiswa terdepan: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[front].tampilkanData();
    }
}

public void tampilkanSemua() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong");
        return;
    }
    System.out.println("Daftar mahasiswa dalam antrian: ");
    System.out.println("NIM - NAMA - PRODI - KELAS");
    for (int i = 0; i < size; i++) {
        int index = (front + i) % max;
        System.out.print((i + 1) + ". ");
        data[index].tampilkanData();
    }
}

public void clear() {
    if (!IsEmpty()) {

```

```

        front = rear = -1;

        size = 0;

        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}

```

```

public void tambahAntrian(Mahasiswa mhs) {
    if (IsFull()) {
        System.out.println("Antrian sudah penuh, tidak dapat menambah mahasiswa.");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + " berhasil masuk ke antrian.");
}

```

```

public Mahasiswa layaniMahasiswa() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return null;
    }
    Mahasiswa mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}

```

```

        public int getJumlahAntrian() {
            return size;
        }
    }
}

```

Class LayananAkademikSIKAD:

```

package Jobsheet10.P2;

import java.util.Scanner;

public class LayananAkademikSIKAD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        AntrianLayanan antrian = new AntrianLayanan(5);
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Akademik ===");
            System.out.println("1. Tambah Mahasiswa Antrian");
            System.out.println("2. Layanan Mahasiswa");
            System.out.println("3. Lihat Mahasiswa Terdepan");
            System.out.println("4. Lihat Semua Antrian");
            System.out.println("5. Jumlah Mahasiswa dalam Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilihan = sc.nextInt();
            sc.nextLine();

            switch (pilihan) {
                case 1:

```

```
System.out.print("NIM    : ");
String nim = sc.nextLine();
System.out.print("Nama    : ");
String nama = sc.nextLine();
System.out.print("Prodi    : ");
String prodi = sc.nextLine();
System.out.print("Kelas    : ");
String kelas = sc.nextLine();
Mahasiswa mhs = new Mahasiswa(nim, nama, prodi, kelas);
antrian.tambahAntrian(mhs);
break;
```

case 2:

```
Mahasiswa dilayani = antrian.layaniMahasiswa();
if (dilayani != null) {
    System.out.print("Melayani mahasiswa: ");
    dilayani.tampilkanData();
}
break;
```

case 3:

```
antrian.lihatTerdepan();
break;
```

case 4:

```
antrian.tampilkanSemua();
break;
```

case 5:

```
System.out.println("Jumlah dalam antrian: " +
antrian.getJumlahAntrian());
```

```

        break;

    case 0:
        System.out.println("Terima kasih.");
        break;

    default:
        System.out.println("Pilihan tidak valid.");
    }
} while (pilihan != 0);
sc.close();
}
}

```

10.3.2 Verifikasi Hasil Percobaan

```

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM      : 123
Nama     : Aldi
Prodi    : TI
Kelas   : 1A
Aldi berhasil masuk ke antrian.

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM      : 124
Nama     : Bobi
Prodi    : TI
Kelas   : 1G
Bobi berhasil masuk ke antrian.

```

```

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Melayani mahasiswa: 123 - Aldi - TI - 1A

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - Bobi - TI - 1G

```

```

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa Antrian
2. Layanan Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 0
Terima kasih.

```

10.3.3 Pertanyaan!

1. Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIAKAD sehingga method LihatAkhir dapat dipanggil!

Jawaban:

Class AntrianLayanan:

```

public void lihatTerbelakang() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.println("Mahasiswa terdepan: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[rear].tampilkanData();
    }
}

```

Class Main:

```

case 6:
    antrian.lihatTerbelakang();
    break;

```

10.4 Tugas

10.4.1 Diagram Class

Mahasiswa10
nim:String nama: String prodi: String

kelas: String
Mahasiswa10)nim: String, nama:String, prodi:String, kelas:String) tampilkanData(): void

AntrianLayanan10
data:Mahasiswa10[] front: int rear: int int size: int max: int maxMhs: int jumlahSudahKRS: int
AntrianLayar10(max:int, maxMhs:int) isEmpty(): boolean isFull(): boolean clear(): void tambahAntrian(mhs: Mahasiswa10): void layaniMahasiswa(): void tampilkanSemua(): void lihatDuaTerdepan(): void lihatTerbelakang(): void getJumlahAntrian(): int getJumlahSudahKRS(): int getJumlahBelumKRS(): int

LayananKRSMain10
tampilMenu(): void tambahData(sc: Scanner, antrian: AntrianLayanan10) : void main(args:String[]): void

10.4.2 Kode Program

Class Mahasiswa:

```
package Jobsheet10.Tugas;
```

```
public class Mahasiswa10 {
    String nim;
    String nama;
    String prodi;
    String kelas;
```

```

        public Mahasiswa10(String nim, String nama, String prodi, String kelas) {
            this.nim = nim;
            this.nama = nama;
            this.prodi = prodi;
            this.kelas = kelas;
        }

        public void tampilkanData() {
            System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
        }
    }

```

Class AntrianLayanan:

package Jobsheet10.Tugas;

```

public class AntrianLayanan10 {
    Mahasiswa10[] data;
    int max;
    int front;
    int rear;
    int size;
    int maxMhs;
    int jumlahSudahKRS;

    public AntrianLayanan10(int max, int maxMhs) {
        this.max = max;
        this.maxMhs = maxMhs;
        data = new Mahasiswa10[max];
        front = 0;
        rear = -1;
        size = 0;
        jumlahSudahKRS = 0;
    }

    public boolean IsEmpty() {
        return size == 0;
    }

    public boolean IsFull() {
        return size == max;
    }

    public void clear() {
        front = 0;
        rear = -1;
        size = 0;
    }
}

```

```

        System.out.println("Antrian berhasil dikosongkan.");
    }

    public void tambahAntrian(Mahasiswa10 mhs) {
        if (IsFull()) {
            System.out.println("Antrian penuh!");
            return;
        }
        rear = (rear + 1) % max;
        data[rear] = mhs;
        size++;
        System.out.println(mhs.nama + " masuk antrian.");
    }

    public void layaniMahasiswa() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }

        int jumlahDiproses = Math.min(2, size);

        System.out.println("Melayani " + jumlahDiproses + " mahasiswa.");
        for (int i = 0; i < jumlahDiproses; i++) {
            Mahasiswa10 mhs = data[front];
            front = (front + 1) % max;
            size--;
            jumlahSudahKRS++;
            mhs.tampilkanData();
        }
    }

    public void tampilkanSemua() {
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }

        System.out.println("Daftar Antrian:");
        for (int i = 0; i < size; i++) {
            int index = (front + i) % max;
            System.out.print((i + 1) + ". ");
            data[index].tampilkanData();
        }
    }

```

```

public void lihatDuaTerdepan() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return;
    }

    System.out.println("2 Antrian Terdepan:");
    for (int i = 0; i < Math.min(2, size); i++) {
        int index = (front + i) % max;
        data[index].tampilkanData();
    }
}

public void lihatTerbelakang() {
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return;
    }

    System.out.println("Antrian Terakhir:");
    data[rear].tampilkanData();
}

public int getJumlahAntrian() {
    return size;
}

public int getJumlahSudahKRS() {
    return jumlahSudahKRS;
}

public int getJumlahBelumKRS() {
    return maxMhs - jumlahSudahKRS;
}
}

Class LayananKRSMMain:
package Jobsheet10.Tugas;

import java.util.Scanner;

public class LayananKRSMMain10 {

    static void tampilMenu() {
        System.out.println("\n=== Menu Antrian Layanan Akademik ===");
    }
}

```

```

        System.out.println("1. Tambah Mahasiswa Antrian");
        System.out.println("2. Proses 2 Mahasiswa");
        System.out.println("3. Tampilkan Semua Antrian");
        System.out.println("4. Lihat 2 Antrian Terdepan");
        System.out.println("5. Lihat Antrian Terakhir");
        System.out.println("6. Jumlah Antrian");
        System.out.println("7. Jumlah Sudah KRS");
        System.out.println("8. Jumlah Belum KRS");
        System.out.println("9. Cek Antrian Kosong");
        System.out.println("10. Cek Antrian Penuh");
        System.out.println("11. Kosongkan Antrian");
        System.out.println("0. Keluar");
        System.out.print("Pilih: ");
    }

    static void tambahData(Scanner sc, AntrianLayanan10 antrian) {
        System.out.print("NIM : ");
        String nim = sc.nextLine();
        System.out.print("Nama : ");
        String nama = sc.nextLine();
        System.out.print("Prodi : ");
        String prodi = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();

        Mahasiswa10 mhs = new Mahasiswa10(nim, nama, prodi, kelas);
        antrian.tambahAntrian(mhs);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan10 antrian = new AntrianLayanan10(10, 30);

        int pilih;

        do {
            tampilMenu();
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    tambahData(sc, antrian);
                    break;
                case 2:
                    antrian.layaniMahasiswa();
            }
        } while (pilih != 0);
    }
}

```

```

        break;
    case 3:
        antrian.tampilkanSemua();
        break;
    case 4:
        antrian.lihatDuaTerdepan();
        break;
    case 5:
        antrian.lihatTerbelakang();
        break;
    case 6:
        System.out.println("Jumlah antrian: " + antrian.getJumlahAntrian());
        break;
    case 7:
        System.out.println("Sudah KRS: " + antrian.getJumlahSudahKRS());
        break;
    case 8:
        System.out.println("Belum KRS: " + antrian.getJumlahBelumKRS());
        break;
    case 9:
        System.out.println(antrian.isEmpty() ? "Kosong" : "Tidak kosong");
        break;
    case 10:
        System.out.println(antrian.IsFull() ? "Penuh" : "Belum penuh");
        break;
    case 11:
        antrian.clear();
        break;
    case 0:
        System.out.println("Terima kasih.");
        break;
    default:
        System.out.println("Pilihan salah!");
}

} while (pilih != 0);

sc.close();
}
}

```

10.4.3 Output

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 3

Daftar Antrian:

1. 1 - g - ti - 1g
2. 2 - k - ti - 1k
3. 3 - o - ti - 1h

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 4

2 Antrian Terdepan:

- 1 - g - ti - 1g
- 2 - k - ti - 1k

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 5

Antrian Terakhir:

3 - o - ti - 1h

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 6

Jumlah antrian: 3

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 7

Sudah KRS: 0

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 8

Belum KRS: 30

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 2

Melayani 2 mahasiswa:

- 1 - g - ti - 1g
- 2 - k - ti - 1k

=== Menu Antrian Layanan Akademik ===

1. Tambah Mahasiswa Antrian
2. Proses 2 Mahasiswa
3. Tampilkan Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Terakhir
6. Jumlah Antrian
7. Jumlah Sudah KRS
8. Jumlah Belum KRS
9. Cek Antrian Kosong
10. Cek Antrian Penuh
11. Kosongkan Antrian
0. Keluar

Pilih: 8

Belum KRS: 28